

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I _____M.Mohammed arsath/192511130_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M.Mohammed arsath

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.
2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

ANSWER:

INTELLIGENT AGENTS IN SMART HOME AUTOMATION SYSTEM

1. INTRODUCTION

A Smart Home Automation System is an intelligent system designed to automatically manage various household activities such as lighting, temperature control, and security. The system uses sensors and intelligent agents to monitor the environment, understand user behavior, and take appropriate actions.

The system must respond to real-time inputs such as:

1. Motion detection
2. Temperature changes
3. Time of day
4. Door and window status
5. User presence
6. Security events
7. User preferences

The system should also learn the user's preferences over time. For example, if a user regularly sets the temperature to 24°C during the night, the system can learn this behavior and automatically adjust the temperature.

The main objectives of the smart home system are:

- Provide user comfort.
- Reduce energy consumption.
- Improve home security.
- Automate routine activities.
- Learn user preferences.
- Respond quickly to environmental changes.

Different types of intelligent agents can be used to implement this system. The major types considered are:

1. Simple Reflex Agent
2. Model-Based Agent
3. Goal-Based Agent
4. Utility-Based Agent

Each type of agent has different capabilities and is suitable for different tasks.

2. INTELLIGENT AGENTS

An intelligent agent is a system that perceives its environment through sensors and performs actions using actuators to achieve a desired objective.

The basic working of an intelligent agent is:

PERCEIVE → ANALYZE → DECIDE → ACT

In a smart home automation system:

Sensors provide information about:

- Motion
- Temperature
- Light level
- Time
- Door and window status
- User presence
- Security events

Actuators perform actions such as:

- Turning lights ON or OFF
- Increasing or decreasing temperature
- Locking or unlocking doors
- Activating alarms
- Controlling fans and air conditioners
- Closing curtains or blinds

The agent receives information from the environment, processes it, and performs appropriate actions.

3. SIMPLE REFLEX AGENT

Definition

A Simple Reflex Agent selects actions based only on the current percept or current condition of the environment.

It uses simple condition-action rules:

IF condition THEN action

The agent does not maintain a history of previous events and does not have an internal model of the environment.

Working in Smart Home Automation

A Simple Reflex Agent can be used for basic automation tasks.

Example 1: Motion Detection

IF motion is detected AND it is night
THEN turn ON the lights.

Example 2: Temperature Control

IF temperature $> 28^{\circ}\text{C}$
THEN turn ON the air conditioner.

Example 3: Security

IF unauthorized motion is detected
THEN activate the security alarm.

Example 4: Lighting

IF room is empty
THEN turn OFF the lights.

The agent immediately responds to the current sensor input.

Advantages

1. Simple to design.
2. Easy to implement.
3. Fast response.
4. Requires less computational power.
5. Suitable for simple automation tasks.

Limitations

1. Does not remember previous situations.
2. Cannot learn user preferences.
3. Cannot handle complex situations.
4. Cannot reason about future consequences.
5. May make incorrect decisions when sensor information is incomplete.

Example

Suppose the temperature sensor detects:

Temperature = 30°C

The Simple Reflex Agent immediately performs:

IF Temperature > 28°C

THEN Turn ON Air Conditioner.

The agent does not consider whether the user is currently at home or whether the air conditioner was recently turned off.

4. MODEL-BASED AGENT

Definition

A Model-Based Agent maintains an internal model or internal state of the environment.

Unlike a Simple Reflex Agent, it remembers information about previous situations and uses that information to make better decisions.

The agent maintains:

CURRENT PERCEPT + INTERNAL STATE → ACTION

The internal state may contain information such as:

- Whether the user is at home.
- Whether the room is occupied.
- Previous temperature.
- Current lighting status.
- Door and window status.
- Security status.

Working in Smart Home Automation

A Model-Based Agent can maintain a model of the home environment.

For example, it can remember:

- The user left home at 9:00 AM.
- The living room is currently empty.
- The front door is locked.
- The temperature is 30°C.
- The bedroom light is OFF.

Based on this internal state, the agent can make better decisions.

Example

Suppose motion is not detected in the house.

A Simple Reflex Agent may turn OFF the lights immediately.

However, a Model-Based Agent checks its internal state and determines whether the user is actually away or simply staying in another room.

It can therefore avoid unnecessary actions.

Advantages

1. Maintains information about previous states.
2. Handles partially observable environments.
3. Makes better decisions using internal knowledge.
4. Can track the current state of the home.
5. More reliable than a Simple Reflex Agent.

Limitations

1. More complex to design.
2. Requires memory.
3. Requires an accurate internal model.
4. May not optimize decisions based on multiple competing objectives.
5. Basic model-based agents may not learn user preferences automatically.

5. GOAL-BASED AGENT

Definition

A Goal-Based Agent selects actions based on a specific goal that it wants to achieve.

The agent considers possible actions and chooses actions that help it reach the desired goal.

The basic process is:

CURRENT STATE → POSSIBLE ACTIONS → GOAL → SELECT ACTION

Working in Smart Home Automation

A Goal-Based Agent can be used to achieve specific smart home goals.

Goal 1: Save Energy

The goal is:

"Minimize unnecessary energy consumption."

The agent may:

- Turn OFF lights in empty rooms.
- Reduce air conditioner usage.
- Turn OFF unused appliances.

Goal 2: Improve Security

The goal is:

"Keep the home secure."

The agent may:

- Lock doors when the user leaves.
- Activate security alarms.
- Monitor unusual movement.

Goal 3: Maintain Comfort

The goal is:

"Maintain a comfortable temperature."

The agent may:

- Turn ON the air conditioner.
- Adjust the thermostat.
- Control fans.

Example

Suppose the user leaves home.

The Goal-Based Agent receives the goal:

"Secure the home and save energy."

The agent may perform:

1. Turn OFF unnecessary lights.

2. Turn OFF the air conditioner.
3. Lock doors.
4. Activate security monitoring.

The agent chooses actions that help achieve the desired goal.

Advantages

1. Can plan actions to achieve specific goals.
2. More flexible than Simple Reflex Agents.
3. Can consider future consequences.
4. Suitable for complex tasks.
5. Can solve multiple automation problems.

Limitations

1. Requires goal definition.
2. Planning may require more computational resources.
3. Does not always determine the best option among multiple successful solutions.
4. May not balance comfort, security, and energy efficiency effectively.

6. UTILITY-BASED AGENT

Definition

A Utility-Based Agent selects an action based on the utility or overall benefit of the outcome. Instead of simply achieving a goal, it tries to choose the best possible outcome among several alternatives.

The agent evaluates different factors such as:

- Comfort
- Energy consumption
- Security
- Cost
- User preferences

The agent selects the action with the highest utility.

The basic process is:

PERCEIVE → EVALUATE OPTIONS → CALCULATE UTILITY → SELECT BEST ACTION → ACT

Working in Smart Home Automation

A Utility-Based Agent is highly suitable for smart home automation because the system must balance multiple objectives.

For example:

- High temperature may require air conditioning.
- Turning ON the air conditioner increases energy consumption.
- The user may prefer a comfortable temperature.
- The system must also consider whether anyone is at home.

The agent evaluates all these factors and selects the best action.

Example

Suppose the temperature is 30°C.

The system has two choices:

Option 1:

Turn ON Air Conditioner.

Benefits:

- High comfort.
- Higher energy consumption.

Option 2:

Turn ON Fan.

Benefits:

- Moderate comfort.
- Lower energy consumption.

The Utility-Based Agent evaluates both options and selects the one with the highest overall utility.

If the user is present and prefers a cool environment, it may select the air conditioner.

If the room is empty, it may select neither and turn OFF both devices.

Utility Evaluation

The system can calculate a utility value using:

UTILITY = Comfort + Security + Energy Efficiency + User Preference

The system may assign different importance or weights to each factor.

For example:

Utility =

$0.4 \times \text{Comfort}$

- $0.3 \times \text{Energy Efficiency}$
- $0.3 \times \text{Security}$

The action with the highest utility is selected.

Advantages

1. Balances multiple objectives.
2. Provides better decision-making.
3. Can optimize comfort and energy efficiency.
4. Can consider user preferences.
5. Suitable for complex real-world environments.
6. Can make decisions when several actions are possible.

Limitations

1. More complex to design.
2. Requires a utility function.
3. Requires accurate sensor data.
4. Utility values may need continuous adjustment.
5. More computationally expensive than simple reflex agents.

7. COMPARISON OF INTELLIGENT AGENTS

Agent Type	Working Principle	Smart Home Application	Advantages	Limitations
Simple Reflex Agent	Uses current percepts and rules	Turn ON lights when motion is detected	Fast and simple	Cannot learn or remember
Model-Based Agent	Uses current percepts and internal state	Track room occupancy and home status	Handles partial information	More complex
Goal-Based Agent	Selects actions to achieve goals	Save energy or improve security	Can plan actions	May not select the best option

Agent Type	Working Principle	Smart Home Application	Advantages	Limitations
Utility-Based Agent	Selects action with highest utility	Balance comfort, security, and energy	Optimizes multiple objectives	Complex to implement

7. ANALYSIS OF EACH AGENT IN THE SMART HOME

8.1 SIMPLE REFLEX AGENT

The Simple Reflex Agent is useful for immediate and straightforward tasks.

Examples:

Motion detected → Turn ON light.

Temperature high → Turn ON fan.

Door opened → Send alert.

It provides fast responses but cannot learn from user behavior.

Therefore, it is suitable for basic automation but not sufficient for the complete smart home system.

8.2 MODEL-BASED AGENT

The Model-Based Agent is useful for maintaining the current state of the home.

For example, it can remember:

- Which rooms are occupied.
- Which lights are ON.
- Whether the user is at home.
- Whether doors are locked.

This helps the system make better decisions even when sensor information is incomplete.

Therefore, it is useful for smart home monitoring and security.

8.3 GOAL-BASED AGENT

The Goal-Based Agent is suitable when the smart home has specific objectives.

Examples:

Goal: Save Energy

→ Turn OFF unused appliances.

Goal: Improve Security

→ Lock doors and activate alarms.

Goal: Maintain Comfort

→ Adjust temperature.

Therefore, it is useful for planning and achieving specific objectives.

8.4 UTILITY-BASED AGENT

The Utility-Based Agent is the most suitable for complex decision-making.

It can balance:

Comfort + Energy Efficiency + Security + User Preferences

For example, if the user is at home, the system may prioritize comfort. If the user leaves, the system may prioritize energy savings and security.

Therefore, the Utility-Based Agent provides better overall decision-making.

9. LEARNING USER PREFERENCES

The smart home system should learn from user behavior over time.

For example, if the user regularly:

- Turns ON bedroom lights at 7:00 PM.
- Sets temperature to 24°C at night.
- Locks doors at 10:00 PM.
- Turns OFF lights when leaving a room.

The system can learn these patterns.

The learning process can be:

OBSERVE USER BEHAVIOR

↓

STORE HISTORICAL DATA

↓

IDENTIFY PATTERNS

↓

LEARN USER PREFERENCES

↓

UPDATE UTILITY VALUES

↓

MAKE PERSONALIZED DECISIONS

For example:

If the user regularly sets the temperature to 24°C at night, the system can automatically set the thermostat to approximately 24°C when nighttime begins.

This improves user comfort and reduces the need for manual control.

10. BEST APPROACH: HYBRID INTELLIGENT AGENT

The most effective solution for the smart home automation system is a **Hybrid Intelligent Agent** that combines the capabilities of all four agent types.

The proposed architecture is:

SENSORS

↓

SIMPLE REFLEX LAYER

↓

MODEL-BASED LAYER

↓

GOAL-BASED PLANNING

↓

UTILITY-BASED DECISION MAKING

↓

LEARNING MODULE

↓

ACTUATORS

11. WORKING OF THE HYBRID APPROACH

Layer 1: Simple Reflex Agent

Handles immediate actions.

Examples:

Motion detected → Turn ON light.

Smoke detected → Activate alarm.

Door opened unexpectedly → Send security alert.

This layer provides fast response to emergencies.

Layer 2: Model-Based Agent

Maintains the internal state of the home.

It tracks:

- User presence.
- Room occupancy.
- Temperature.
- Lighting status.
- Door status.
- Security status.

This allows the system to understand the current situation.

Layer 3: Goal-Based Agent

Manages specific goals.

Examples:

- Maintain comfortable temperature.
- Save energy.
- Secure the house.
- Provide automatic lighting.

The system plans actions to achieve these goals.

Layer 4: Utility-Based Agent

Balances multiple objectives.

It evaluates:

- Comfort.
- Energy efficiency.
- Security.
- Cost.
- User preferences.

The agent selects the action that provides the highest overall utility.

Layer 5: Learning Module

The learning component observes user behavior and updates the system's knowledge.

It learns:

- Preferred temperature.
- Preferred lighting.
- Daily routines.
- Security patterns.
- Energy usage patterns.

This makes the smart home more personalized over time.

12. EXAMPLE OF HYBRID AGENT DECISION-MAKING

Consider the following situation:

Time = 10:00 PM

User = At Home

Temperature = 28°C

Motion = Detected

Door = Locked

User Preference = 24°C

The hybrid agent processes the information.

Simple Reflex Layer:

Motion is detected.

→ Keep the lights ON.

Model-Based Layer:

User is present in the room.

→ Room is occupied.

Goal-Based Layer:

Goal = Maintain user comfort.

→ Temperature must be reduced.

Utility-Based Layer:

Compare:

Option 1: Air Conditioner

Option 2: Fan

Option 3: No Action

The system evaluates comfort and energy consumption.

Since the user prefers 24°C, the system may turn ON the air conditioner and set the temperature to 24°C.

Learning Layer:

The system records:

At 10:00 PM → User prefers 24°C.

Over time, the system learns this pattern and can automatically adjust the temperature in the future.

13. WHY THE HYBRID APPROACH IS MOST EFFECTIVE

A hybrid intelligent agent is the most effective approach because no single agent type can efficiently handle all smart home requirements.

Simple Reflex Agent

Provides fast responses.

Model-Based Agent

Maintains knowledge about the home.

Goal-Based Agent

Plans actions to achieve specific objectives.

Utility-Based Agent

Balances comfort, energy efficiency, and security.

Learning Module

Adapts to user behavior and preferences.

Together, these components create a more intelligent and adaptive smart home system.

14. BENEFITS OF THE HYBRID APPROACH

1. Improved Comfort

The system learns user preferences and automatically adjusts:

- Temperature.
- Lighting.
- Room conditions.

2. Energy Efficiency

The system avoids unnecessary energy consumption by:

- Turning OFF unused lights.
- Adjusting air conditioning.
- Detecting empty rooms.
- Optimizing appliance usage.

3. Enhanced Security

The system can:

- Detect unauthorized movement.
- Lock doors automatically.
- Activate alarms.
- Send security notifications.
-

4. Fast Response

Simple Reflex rules allow the system to respond immediately to emergency situations.

5. Intelligent Decision-Making

Utility-based reasoning helps the system balance competing requirements.

6. Personalization

The learning component adapts the system to individual user preferences.

7. Adaptability

The system can change its behavior based on changing environmental conditions and user behavior.

15. PERFORMANCE MEASURE

The smart home automation system can be evaluated based on:

1. User comfort.
2. Energy consumption.
3. Security level.
4. Response time.
5. Accuracy of user preference prediction.
6. Reduction in unnecessary energy usage.
7. Number of security incidents detected.
8. User satisfaction.

The ideal system should:

MAXIMIZE COMFORT

•

MAXIMIZE SECURITY

•

MAXIMIZE ENERGY EFFICIENCY

•

MAXIMIZE USER SATISFACTION

CONCLUSION

A Smart Home Automation System requires intelligent decision-making because it must respond to real-time environmental conditions while also understanding user behavior and preferences.

A **Simple Reflex Agent** is suitable for immediate rule-based actions such as turning ON lights when motion is detected. A **Model-Based Agent** maintains an internal state of the home and is useful when the environment is partially observable. A **Goal-Based Agent** plans actions to achieve specific objectives such as saving energy or improving security. A **Utility-Based Agent** evaluates multiple alternatives and selects the action that provides the highest overall benefit.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

ANSWER:

SOLVING AN UNKNOWN MAZE USING UNINFORMED SEARCH STRATEGIES

1. INTRODUCTION

A robot is placed in an unknown maze and must find a path from its starting position to the exit. The robot has no prior knowledge of the maze layout and must explore the environment to discover the correct path.

The robot must make decisions based on the information available during exploration. It may encounter:

- Blocked paths.
- Dead ends.
- Multiple possible routes.
- Unknown areas.
- Different path costs.

Since the robot has no prior information about the maze, it cannot directly select the correct path to the exit. Therefore, it must use a search strategy to systematically explore the maze.

Two important uninformed search strategies that can be used are:

1. Uniform Cost Search (UCS)
2. Iterative Deepening Search (IDS)

These algorithms do not use domain-specific heuristic information to guide the search.

Instead, they explore the search space systematically according to their own search principles.

The problem can also be analyzed using different types of intelligent agents, such as:

- Simple Reflex Agent
- Model-Based Agent
- Goal-Based Agent
- Utility-Based Agent

The choice of agent design directly affects how the robot perceives the maze, makes decisions, remembers explored paths, and selects actions.

2. PROBLEM FORMULATION

The unknown maze problem can be represented as a search problem.

Initial State

The robot's starting position in the maze.

Represented as:

S = Start

Goal State

The exit of the maze.

Represented as:

G = Goal

States

Each position or cell in the maze represents a state.

For example:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

Each cell represents a possible position of the robot.

Actions

The robot may perform actions such as:

- Move Up
- Move Down
- Move Left
- Move Right

The robot cannot move through walls or blocked paths.

Transition Model

When the robot performs a valid movement action, it moves from the current position to an adjacent accessible position.

Path Cost

The cost of a path is the total cost of all movements made by the robot.

If every movement has a cost of 1:

Total Path Cost = Number of Movements

If different paths have different costs, the robot must consider the cumulative cost.

Goal Test

The search is successful when:

Current Position = Exit (G)

3. UNINFORMED SEARCH STRATEGIES

Uninformed search strategies are also called **Blind Search Strategies**.

They do not have additional information about how close a state is to the goal.

They use only:

- Initial state.
- Available actions.
- Transition model.
- Goal test.
- Path cost.

In the unknown maze problem, the robot does not know the complete maze layout. Therefore, it must explore the maze systematically.

Two suitable uninformed search algorithms are:

1. Uniform Cost Search (UCS)
2. Iterative Deepening Search (IDS)

4. UNIFORM COST SEARCH (UCS)

Definition

Uniform Cost Search is an uninformed search algorithm that always expands the node with the lowest cumulative path cost.

The cumulative path cost is represented by:

$g(n)$ = Total Cost from Start to Node n

UCS uses a **Priority Queue**.

The node with the smallest cumulative cost is selected for expansion.

The basic process is:

START

↓

Calculate Path Costs

↓
Select Lowest-Cost Node
↓
Expand Node
↓
Add New Nodes to Priority Queue
↓
Select Lowest-Cost Node
↓
Continue Until Goal is Found

5. UCS IN THE UNKNOWN MAZE

Suppose the robot starts at S.

The robot initially knows only the starting position.

It explores neighboring cells and adds them to the priority queue.

For example:

START = S

Possible paths:

$S \rightarrow A$

$S \rightarrow B$

If:

$\text{Cost}(S \rightarrow A) = 2$

$\text{Cost}(S \rightarrow B) = 5$

UCS selects:

$S \rightarrow A$

because its cumulative cost is lower.

The robot continues exploring the lowest-cost path.

If it reaches a dead end, it returns to the search process and explores another available path.

The search continues until the robot reaches the exit.

6. ADVANTAGES OF UCS

1. Optimal

UCS finds the least-cost path when all step costs are non-negative.

Therefore, if different paths have different costs, UCS can find the cheapest route.

2. Complete

UCS is complete when the path costs are positive and a solution exists.

3. Suitable for Variable Path Costs

If different maze paths have different costs, UCS can consider the total cost.

4. Systematic Search

The algorithm systematically explores the search space based on cumulative cost.

7. DISADVANTAGES OF UCS

1. High Memory Usage

UCS stores many nodes in the priority queue.

Therefore, its memory requirement can become very large.

2. Slow for Large Search Spaces

If many paths have similar costs, UCS may explore a large number of nodes before finding the goal.

3. Requires Priority Queue

The implementation is more complex than simple BFS or DFS because nodes must be ordered according to path cost.

4. Does Not Use Goal Direction

UCS does not know which direction leads toward the exit.

Therefore, it may explore many unnecessary paths.

8. UCS TIME AND SPACE COMPLEXITY

Let:

b = Branching factor

C^* = Cost of the optimal solution

ϵ = Minimum step cost

Time Complexity

The time complexity of UCS is approximately:

$$O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$$

In the worst case, UCS may explore a very large number of nodes before reaching the goal.

Space Complexity

The space complexity is also approximately:

$$O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$$

This is because UCS stores generated nodes in the priority queue.

Therefore, UCS can be expensive in terms of memory.

9. ITERATIVE DEEPENING SEARCH (IDS)

Definition

Iterative Deepening Search combines the advantages of Depth-First Search and Breadth-First Search.

IDS repeatedly performs Depth-Limited Search with increasing depth limits.

The search is performed as:

Depth Limit = 0

↓

Depth Limit = 1

↓

Depth Limit = 2

↓

Depth Limit = 3

↓

Continue Until Goal is Found

The robot gradually increases the search depth until it reaches the maze exit.

10. IDS IN THE UNKNOWN MAZE

Suppose the robot starts at S.

Iteration 1

Depth Limit = 0

Search:

S

Goal not found.

Iteration 2

Depth Limit = 1

Search:

$S \rightarrow A$

$S \rightarrow B$

Goal not found.

Iteration 3

Depth Limit = 2

Search all nodes up to depth 2.

The robot explores:

$S \rightarrow A \rightarrow C$

$S \rightarrow A \rightarrow D$

$S \rightarrow B \rightarrow E$

$S \rightarrow B \rightarrow F$

Goal not found.

Iteration 4

Depth Limit = 3

The robot searches all nodes up to depth 3.

If the exit is found at this depth:

Search terminates.

Therefore, IDS gradually increases the depth limit until the goal is reached.

11. ADVANTAGES OF IDS

1. Low Memory Requirement

IDS performs depth-limited DFS and stores only the current path and a limited number of nodes.

Therefore, it requires less memory.

2. Complete

IDS is complete when the branching factor is finite.

3. Optimal for Equal Step Costs

If every movement has the same cost, IDS can find the shallowest solution.

4. Suitable for Unknown Depth

The robot does not need to know how far the exit is.

The depth limit is increased automatically.

12. DISADVANTAGES OF IDS

1. Repeated Exploration

The same nodes may be explored multiple times at different depth limits.

This increases computation time.

2. Not Suitable for Different Path Costs

IDS considers depth rather than cumulative cost.

Therefore, if different paths have different costs, it may not find the least-cost path.

3. Can Be Slow

Repeated searches may increase the total execution time.

4. No Cost Awareness

IDS does not consider the actual cost of individual movements.

It mainly considers the depth of the solution.

13. IDS TIME AND SPACE COMPLEXITY

Let:

b = Branching factor

d = Depth of the shallowest goal

Time Complexity

The time complexity of IDS is:

$$O(b^d)$$

Although nodes at smaller depths are repeatedly explored, the overall complexity remains approximately $O(b^d)$.

Space Complexity

The space complexity is:

$$O(bd)$$

This is much lower than UCS because IDS stores only a limited number of nodes at each depth.

14. COMPARISON BETWEEN UCS AND IDS

Feature	Uniform Cost Search (UCS)	Iterative Deepening Search (IDS)
Search Type	Uninformed	Uninformed
Main Strategy	Expands lowest-cost node	Repeated depth-limited search
Data Structure	Priority Queue	Stack / DFS
Complete	Yes, with positive costs	Yes, with finite branching
Optimal	Yes, for non-negative costs	Yes, when step costs are equal
Time Complexity	$O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$	$O(b^d)$
Space Complexity	$O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$	$O(bd)$
Memory Usage	High	Low
Handles Different Costs	Yes	No
Repeated Exploration	Less repetition	High repetition
Best Use	Variable-cost paths	Equal-cost paths and limited memory

15. TIME AND SPACE ANALYSIS

Uniform Cost Search

Time

UCS can take a long time because it may explore all paths with costs less than the optimal path cost.

Therefore:

$$\text{Time Complexity} \approx O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$$

Space

UCS stores many generated nodes.

Therefore:

$$\text{Space Complexity} \approx O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$$

Thus, UCS has high memory requirements.

Iterative Deepening Search

Time

IDS repeatedly searches the maze with increasing depth limits.

Therefore:

$$\text{Time Complexity} = O(b^d)$$

Space

IDS stores only the current search path and limited nodes.

Therefore:

Space Complexity = $O(bd)$

Thus, IDS uses significantly less memory than UCS.

16. RELATION TO INTELLIGENT AGENTS

The robot can be designed using different types of intelligent agents.

The choice of agent type affects how the robot makes decisions in the unknown maze.

The four major agent types are:

1. Simple Reflex Agent
2. Model-Based Agent
3. Goal-Based Agent
4. Utility-Based Agent

17. SIMPLE REFLEX AGENT

A Simple Reflex Agent selects actions based only on the current percept.

It follows condition-action rules.

For example:

IF front path is blocked

THEN turn right.

IF right path is blocked

THEN turn left.

The robot reacts immediately to the current situation.

Advantages

- Simple.
- Fast.
- Easy to implement.

Limitations

- Does not remember previous locations.
- May get stuck in loops.
- Cannot plan a complete route.
- Not suitable for complex unknown mazes.

Therefore, a Simple Reflex Agent alone is not suitable for solving a large unknown maze.

18. MODEL-BASED AGENT

A Model-Based Agent maintains an internal representation of the environment.

The robot can remember:

- Visited cells.
- Blocked paths.
- Explored routes.
- Current position.
- Previously discovered areas.

For example, if the robot discovers a blocked path at position A, it records the information in its internal map.

When it encounters A again, it knows that the path is blocked.

Advantages

- Maintains memory.
- Avoids repeated exploration.
- Handles partially observable environments.
- Can build a map of the maze.

Limitations

- Requires additional memory.
- More complex than a Simple Reflex Agent.

A Model-Based Agent is highly suitable for an unknown maze.

19. GOAL-BASED AGENT

A Goal-Based Agent selects actions based on a specific goal.

The robot's goal is:

REACH THE EXIT

The agent evaluates possible paths and selects actions that help it reach the exit.

For example:

START

↓

EXPLORE PATH

↓

CHECK FOR EXIT

↓

IF NOT FOUND

↓

EXPLORE ANOTHER PATH

↓

REPEAT

↓

REACH EXIT

A Goal-Based Agent is suitable because the robot has a clear objective.

20. UTILITY-BASED AGENT

A Utility-Based Agent selects actions based on the utility or quality of the outcome.

The robot may consider:

- Path length.
- Travel cost.
- Safety.
- Energy consumption.
- Time.

For example, if two paths lead to the exit:

Path A:

Shorter but risky.

Path B:

Longer but safer.

The Utility-Based Agent evaluates both paths and selects the one with higher utility.

If the maze has different movement costs, a Utility-Based Agent combined with UCS can make better decisions.

21. HOW AGENT DESIGN AFFECTS DECISION-MAKING

The agent architecture directly affects the robot's ability to solve the maze.

Simple Reflex Agent

Decision based on:

CURRENT PERCEPT

It reacts to immediate situations.

Model-Based Agent

Decision based on:

CURRENT PERCEPT + INTERNAL STATE

It remembers previously explored areas.

Goal-Based Agent

Decision based on:

CURRENT STATE + GOAL

It chooses actions that help reach the exit.

Utility-Based Agent

Decision based on:

CURRENT STATE + GOAL + UTILITY

It chooses the best action based on cost, safety, time, and other factors.

Therefore, as the agent architecture becomes more advanced, the robot can make more intelligent and informed decisions.

22. MOST EFFECTIVE COMBINATION

The most effective combination for solving an unknown maze is:

MODEL-BASED GOAL-BASED AGENT + UNIFORM COST SEARCH

This combination is highly suitable when:

- The maze is initially unknown.
- The robot must remember explored areas.
- Different paths may have different costs.
- The robot must reach a specific goal.
- The robot needs to avoid unnecessary exploration.

23. WHY MODEL-BASED AGENT IS SUITABLE

The robot does not initially know the complete maze.

Therefore, it needs an internal map.

The Model-Based Agent can store:

- Visited locations.
- Obstacles.
- Dead ends.
- Available paths.
- Current position.

This prevents the robot from repeatedly exploring the same blocked paths.

24. WHY GOAL-BASED AGENT IS SUITABLE

The robot has a clear goal:

REACH THE MAZE EXIT

The Goal-Based Agent helps the robot select actions that move it toward achieving the goal.

It can work together with the Model-Based Agent to use the known information about the maze.

25. WHY UCS IS SUITABLE

UCS is the best choice when path costs are not necessarily equal.

For example:

Path A = 10 cost units

Path B = 15 cost units

Path C = 8 cost units

UCS explores the path with the lowest cumulative cost.

Therefore, it can find the least-cost path to the exit.

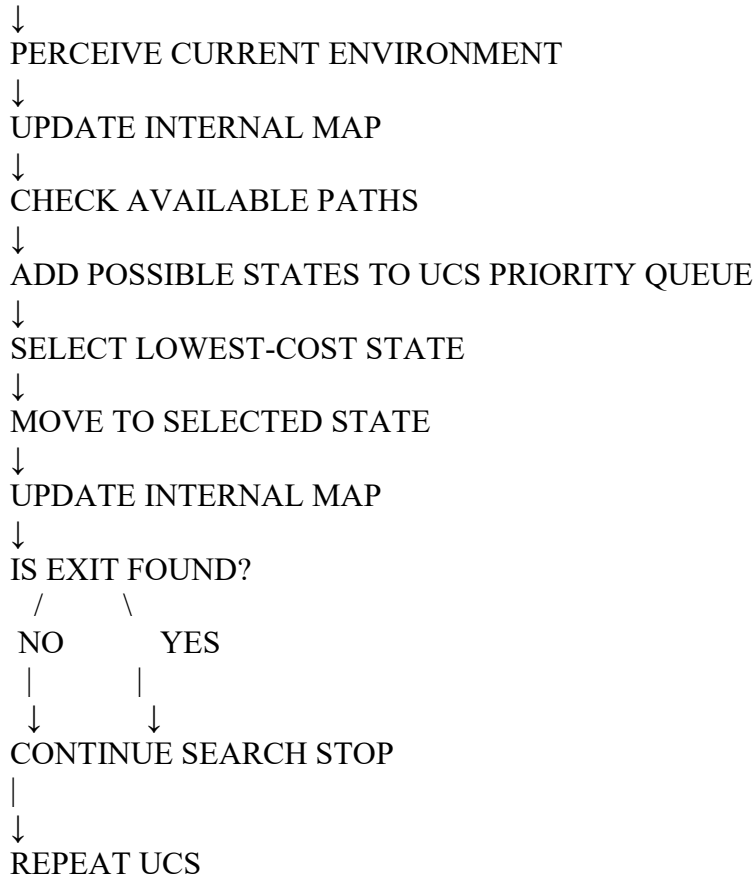
UCS is especially suitable when the robot must minimize:

- Travel cost.
- Energy consumption.
- Movement cost.
- Risk cost.

26. OVERALL WORKING OF THE PROPOSED SYSTEM

The complete system can be represented as:

ROBOT STARTS AT S



27. EXAMPLE OF DECISION-MAKING

Suppose the robot reaches a position where it has three possible paths.

Path A:

Cost = 3

Path B:

Cost = 5

Path C:

Cost = 8

The Model-Based Agent checks its internal map.

It knows:

Path A → Previously explored and leads to a dead end.

Path B → Unexplored.

Path C → Previously explored.

The Goal-Based Agent identifies that the goal is still not reached.

UCS evaluates the available paths based on cumulative cost.

The robot selects:

Path B

because it provides a valid unexplored route with a reasonable cost. This demonstrates how agent design and search strategy work together to improve decision-making.

28. FINAL RECOMMENDATION

The recommended solution is:

MODEL-BASED + GOAL-BASED INTELLIGENT AGENT

combined with:

UNIFORM COST SEARCH (UCS)

Reason:

The Model-Based Agent allows the robot to remember the maze structure as it explores.

The Goal-Based Agent provides a clear objective of reaching the exit.

UCS selects the path with the minimum cumulative cost.

Together, they allow the robot to:

- Explore an unknown maze.
- Remember visited locations.
- Avoid known obstacles.
- Avoid unnecessary repeated exploration.
- Consider different path costs.
- Find the least-cost path.
- Reach the exit efficiently.

If all movements have equal cost and memory is severely limited, **IDS** can be used as an alternative because it requires much less memory. However, if the maze contains different path costs or the robot must minimize travel cost, **UCS is the preferred choice**.

CONCLUSION

An autonomous robot placed in an unknown maze must systematically explore the environment to find the exit. Since the robot has no prior knowledge of the maze layout, uninformed search strategies such as Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can be used.

UCS expands the node with the lowest cumulative path cost and is complete and optimal when path costs are non-negative. It is suitable when different paths have different costs, but it requires high memory and can be computationally expensive.

IDS performs repeated depth-limited searches with increasing depth limits. It uses much less memory and is complete and optimal for equal step costs, but it repeatedly explores the same nodes and does not consider different path costs.

The problem can also be related to different intelligent agent types. A Simple Reflex Agent reacts only to the current situation and is not suitable for complex maze navigation. A Model-Based Agent maintains an internal map of the environment and can remember explored paths. A Goal-Based Agent makes decisions based on the objective of reaching the exit. A Utility-Based Agent can consider additional factors such as cost, safety, energy, and time.

The most effective solution is a **Model-Based Goal-Based Agent combined with Uniform Cost Search (UCS)**. The Model-Based Agent helps the robot remember the maze as it explores, while the Goal-Based Agent focuses on reaching the exit. UCS ensures that the robot selects the least-cost path when different movement costs exist.

Therefore, the combination of a **Model-Based and Goal-Based Intelligent Agent with Uniform Cost Search** provides an effective solution for navigating an unknown maze. It allows the robot to make informed decisions, maintain knowledge of the environment, avoid unnecessary exploration, and reach the exit using an optimal or least-cost path.